SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5

4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5

9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

Code of Conduct:

*I **Dhaanush Rajan(192421162)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

ANSWER

Question 1 – Reverse Engineering an ATM Withdrawal Module

1.1 System understanding and illustrative assumptions

The question states that only ATM withdrawal input-output behavior is known, including withdrawal limits, balance validation, and error messages. No actual limit or observed response table is supplied. For demonstration only, assume a minimum withdrawal of ₹100, a maximum per transaction of ₹20,000, withdrawals in multiples of ₹100, and a PIN authentication step. These values are examples, not facts from the assessment.

1.2 Functional decomposition

Component	Responsibility	Observable checks
Card/session validation	Checks card status and starts a session	Valid/expired/blocked card; session timeout
PIN validation	Authenticates user	Correct, incorrect, empty PIN; retry lockout
Amount validation	Checks numeric amount, denomination, minimum and maximum	Zero, negative, non-numeric, non-multiple, limits
Balance validation	Checks available balance and holds	Below, equal to, and above available balance
Daily/transaction limit	Applies configured limits	At limit and over limit
Cash dispenser	Dispenses requested notes when cash is available	Sufficient/insufficient ATM cash; denomination availability
Account update	Debits account after successful dispense	Exactly-once debit; rollback on dispense failure
Receipt/audit log	Records outcome and communicates status	Correct message, masked account data, audit entry

1.3 Equivalence classes and boundaries

Input / condition	Equivalence class	Representative tests
Amount	Invalid: non-numeric, blank, negative, zero	abc; blank; −100; 0
Amount	Below assumed minimum	₹0, ₹99
Amount	Valid amount range and denomination	₹100, ₹500, ₹19,900
Amount	Not a multiple of ₹100	₹150

Amount	Above assumed maximum	₹20,001; ₹25,000
Available balance	Less than requested amount	Balance ₹400, request ₹500
Available balance	Equal to requested amount	Balance ₹500, request ₹500
Available balance	Greater than requested amount	Balance ₹600, request ₹500
Card/PIN	Invalid or blocked authentication	Expired card; wrong PIN at retry threshold
ATM cash	Insufficient notes/cash	Account can cover amount but ATM cannot dispense it

1.4 Likely original black-box techniques and adequacy

If the partial tests cover a normal withdrawal, an insufficient-balance message, and a maximum limit error, the likely methods are basic functional testing, ECP, and perhaps BVA. This is an inference only; the question does not provide actual test cases from which the original technique can be proven.

- Strength: externally visible rules can be tested without source code.
- Likely gap: testing only typical values misses off-by-one errors at minimum, maximum, daily limits, and available balance.
- Likely gap: amount validation may be tested independently but not combined with ATM cash shortage, blocked card, or concurrent account activity.
- Likely gap: error messages may be checked without verifying account balance, audit log, or rollback side effects.

1.5 Reconstructed improved test suite

ID	Test condition / input	Expected result
ATM-01	Valid card/PIN; balance ₹5,000; request ₹500	Cash dispensed; balance decreases exactly ₹500; success message and audit record
ATM-02	Request ₹99 (assumed minimum ₹100)	Rejected as below minimum; no debit
ATM-03	Request ₹100	Accepted if other conditions pass
ATM-04	Request ₹20,000 (assumed maximum) with sufficient balance/cash	Accepted if daily limit permits
ATM-05	Request ₹20,001	Rejected as above per-transaction limit; no debit
ATM-	Request ₹150 (not a ₹100 multiple)	Rejected with clear denomination/multiple

06		message
ATM-07	Balance ₹499; request ₹500	Insufficient funds; no cash and no debit
ATM-08	Balance exactly ₹500; request ₹500	Successful if no other limit applies; ending balance ₹0
ATM-09	Correct amount but incorrect PIN	Rejected; balance unchanged; retry counter updated
ATM-10	Card blocked/expired	Transaction refused before withdrawal; no debit
ATM-11	Sufficient account balance but ATM has insufficient cash	Clear failure message; no account debit
ATM-12	Cash dispenser fails after authorization	Transaction is reversed/rolled back; audit trail records failure
ATM-13	Repeated request after network timeout	No duplicate debit or duplicate cash dispense
ATM-14	Daily limit reached; otherwise valid request	Rejected according to configured daily-limit rule
ATM-15	Blank, alphabetic, extremely long, or malformed amount field	Input safely rejected; no crash or state change

1.6 Conclusion

The improved design combines ECP and BVA with state, negative, integration, and transaction-integrity tests. Adequacy should be measured against every documented withdrawal rule, each valid/invalid class, all defined boundaries, and side effects such as balance consistency and rollback—not merely the number of tests.

Question 2 – Reverse Engineering White-Box Testing from Traces and Code

2.1 Evidence limitation and illustrative program

The question refers to execution traces and code snippets, but none are included in the supplied assessment pages. The following small pseudocode example is therefore illustrative and demonstrates how to construct a control flow graph (CFG), decisions, paths, and coverage criteria. It is not a reconstruction of unseen source code.

```
1  read amount, balance
2  if amount <= 0: return INVALID_AMOUNT
3  if amount > balance: return INSUFFICIENT_FUNDS
4  attempts = 0
5  while attempts < 2:
6      if account_is_active():
7          debit(amount); return SUCCESS
8      attempts = attempts + 1
9  return ACCOUNT_UNAVAILABLE
```

2.2 Control flow graph

Conceptual CFG (each numbered node represents a statement or decision):

START → 1 → 2 [amount ≤ 0?]
2-YES → INVALID_AMOUNT → END
2-NO → 3 [amount > balance?]
3-YES → INSUFFICIENT_FUNDS → END
3-NO → 4 → 5 [attempts < 2?]
5-NO → ACCOUNT_UNAVAILABLE → END
5-YES → 6 [account active?]
6-YES → 7 SUCCESS → END
6-NO → 8 increment attempts → back to 5

2.3 Decisions, loops and independent paths

Element	Analysis
Decision 2	Amount ≤ 0: true and false outcomes
Decision 3	Amount > balance: true and false outcomes
Loop decision 5	attempts < 2: enter/repeat and exit outcomes
Decision 6	Account active: true and false outcomes
Loop	At most two attempts; loop has zero, one, or two active-check iterations depending on control flow

Independent-path estimate	For this illustrative CFG, cyclomatic complexity is approximately E − N + 2P. With one connected component, it equals number of decision nodes + 1; four binary decisions imply V(G)=5, subject to CFG node/edge conventions.

2.4 Example basis paths and tests

Path	Input / stub condition	Expected outcome
P1: invalid amount exit	amount = 0	INVALID_AMOUNT; balance unchanged
P2: insufficient funds exit	amount = 500, balance = 100	INSUFFICIENT_FUNDS
P3: first account check succeeds	amount = 100, balance = 500; active on first check	SUCCESS; exactly one debit
P4: first check fails, second succeeds	amount = 100, balance = 500; inactive then active	SUCCESS; exactly one debit; two checks
P5: both checks fail	amount = 100, balance = 500; inactive twice	ACCOUNT_UNAVAILABLE; no debit

2.5 Coverage adequacy evaluation

- Statement coverage: every statement is executed at least once. It can miss a decision outcome when statements on only one side are reached.
- Branch coverage: each decision takes true and false outcomes at least once. This is stronger than statement coverage for the shown decisions.
- Condition coverage: each atomic Boolean condition is independently true and false; relevant for compound conditions.
- Basis-path testing: select a linearly independent set of paths, often guided by cyclomatic complexity.
- Loop testing: test zero iterations where possible, one iteration, typical iterations, maximum permitted iterations, and termination/failure.

If the provided execution traces (when available) show only successful paths or never exercise false outcomes, adequacy is incomplete. Because actual traces are absent, no factual coverage percentage can be calculated.

2.6 Reconstructed strategy

1. Obtain the actual source snippets and normalize each statement and decision into CFG nodes and edges.
2. Mark every decision outcome and loop back-edge; identify unreachable or infeasible edges.
3. Compute cyclomatic complexity using V(G)=E−N+2P, then select a basis set of independent paths.
4. Create tests that force each branch and loop boundary using controlled inputs and stubs.
5. Instrument the module and collect statement, branch, condition, and path evidence.
6. Report uncovered edges and explain any infeasible paths rather than claiming coverage without evidence.

Question 3 – Web Application Login and Transaction System

3.1 Reverse-engineering defect patterns

The question supplies the existence of defect logs and test results, but no actual entries. Accordingly, the mappings below are examples of how an analyst would interpret common defect patterns; they are not claims about the unseen logs.

Illustrative defect pattern	Likely missed technique / weakness	Follow-up evidence to inspect
Login fails only for empty or very long username	ECP/BVA or input validation gap	Input lengths, error codes, boundary values
Valid login works but locked account can transact	State/authorization testing gap	Account status, session and authorization checks
Transaction fails at exact balance or limit	BVA or numeric precision gap	Values at limit−1, limit, limit+1
Duplicate debit after retry/timeout	Integration, idempotency, and path testing gap	Request IDs, retries, transaction states
Only one outcome of an if/else branch is logged	White-box branch coverage gap	Coverage report and control-flow trace
Unauthorised user can access another account	Role/ownership negative testing gap	Identity-to-account authorization checks

3.2 Functional decomposition

- Login and identity verification: username/password format, credential verification, account status, lockout and session creation.
- Session management: expiry, logout, concurrent sessions, cookie/token validity and privilege changes.
- Transaction request validation: amount, account identifiers, transaction type, currency and required fields.
- Authorization: role, ownership, available funds, daily limits and transaction permissions.
- Transaction processing: debit/credit, atomicity, duplicate prevention, reversal and timeout handling.
- Audit and notification: correct status, traceable event, safe error messages and no sensitive-data leakage.

3.3 Reconstructed integrated strategy

Technique	How it is applied	Representative test
ECP	Group credentials and transaction inputs into valid and invalid classes	Valid credentials; empty password; unknown user; malformed amount
BVA	Probe lengths, retry count, transaction limits and balance limits	Password length min−1/min/min+1; amount limit−1/limit/limit+1
Decision table	Combine account state, authentication, role, balance and limit conditions	Authenticated + active + authorized + sufficient funds → allow

State-transition testing	Exercise login/session and transaction status changes	Logged out → authenticated → expired; Pending → Success/Reversal
White-box branch/path testing	Use source/coverage data to force both outcomes of authorization and transaction decisions	Owner/non-owner, within/over limit, success/failure of commit
Negative/security testing	Probe invalid tokens, injection strings, replay and unauthorized access	Tampered account ID; expired token; duplicate transaction ID

3.4 Decision table for transaction authorization

Rule	Authenticated?	Account active?	Authorized owner?	Sufficient funds & within limit?	Expected action
R1	Yes	Yes	Yes	Yes	Process transaction
R2	No	—	—	—	Reject; request authentication
R3	Yes	No	—	—	Reject inactive account
R4	Yes	Yes	No	—	Reject authorization
R5	Yes	Yes	Yes	No	Reject; no debit
R6	Yes	Yes	Yes	Yes, but commit fails	Rollback/reversal and record failure

3.5 Coverage gaps and validation

A good reconstruction should connect each defect to a testable hypothesis, then create a test that would reproduce or disprove it. Confirm that all externally defined input classes and boundaries are tested; all transaction states and legal/illegal transitions are exercised; and branch coverage demonstrates both outcomes of critical decisions. Security tests should be conducted only in an authorized test environment.

3.6 Conclusion

Combining black-box partitions and boundaries with white-box paths gives complementary evidence: black-box tests establish externally correct behavior, while path and branch tests reveal unexecuted internal logic. Actual defect-log analysis remains pending until the logs are supplied.

Question 4 – Legacy Inventory Management System

4.1 System decomposition and inferred rules

The assessment describes successful/failed stock-update logs and low-stock alerts, but provides no rows or threshold values. For illustration only, assume stock must never become negative, a reorder alert is raised when stock is at or below a configurable threshold of 10 units, and updates accept integer quantities. The actual system may use different rules.

- Input/API validation: item ID, quantity, operation type and data type.
- Stock calculation: add received stock or subtract issued stock.
- Constraint enforcement: reject negative final stock and invalid quantities.
- Alert engine: compare new stock with reorder threshold and avoid missing/duplicating alerts.
- Persistence and concurrency: commit stock change atomically and avoid lost updates.
- Audit/reporting: record before/after quantities, user, timestamp and outcome.

4.2 Equivalence classes and boundary conditions

Field / rule	Equivalence partitions	Boundary-focused tests
Quantity	Valid positive integer; zero; negative; non-integer; null	−1, 0, 1; fractional quantity if disallowed
Current stock	Zero; positive; corrupted negative value	0 and 1; invalid stored state handling
Issue operation	Quantity less than stock; equal to stock; greater than stock	stock−1, stock, stock+1
Reorder threshold	Stock above threshold; equal; below	threshold+1, threshold, threshold−1
Item identifier	Existing; nonexistent; malformed; inactive item	Valid ID, unknown ID, blank ID
Concurrent update	Sequential; simultaneous issue/receipt	Two issues competing for same remaining stock

4.3 Illustrative log interpretation

Hypothetical observation	Possible hidden rule	Test needed
Issue succeeds when stock exactly equals issue quantity	System permits final stock = 0	Stock 5, issue 5; verify zero and no negative value
Issue fails when quantity exceeds stock by one	Availability constraint exists	Stock 5, issue 6; verify no mutation
Alert appears at stock 10 but not 11	Threshold may be inclusive	Test 9, 10 and 11; verify exact configured rule
Receipt reduces stock in the	Possible sign/operation defect	Receipt 5 to stock 10 should

log		normally increase to 15
Two simultaneous issues result in negative stock	Concurrency/locking defect	Parallel requests against limited stock

4.4 Reconstructed black-box test cases

ID	Setup and action	Expected result
INV-01	Existing item stock 10; receive 5	Stock becomes 15; transaction logged
INV-02	Stock 5; issue 4	Stock becomes 1; success
INV-03	Stock 5; issue 5	Stock becomes 0 if zero stock is permitted; alert behavior follows configured threshold
INV-04	Stock 5; issue 6	Rejected; stock remains 5; clear insufficient-stock response
INV-05	Receive quantity 0	Rejected or handled according to documented no-op policy; no accidental mutation
INV-06	Issue negative quantity −3	Rejected; must not increase stock by treating negative issue as receipt
INV-07	Stock changes from 11 to 10	Alert is generated if threshold is inclusive at 10
INV-08	Stock changes from 10 to 9	Alert remains active or updates according to alert policy; no duplicate spam
INV-09	Unknown item ID	Not found; no stock record created unintentionally
INV-10	Two concurrent issues each request 4 while stock is 5	At most one or a valid combination succeeds; final stock cannot be negative
INV-11	Database commit fails after validation	Update is rolled back; no misleading success or alert
INV-12	Repeat the same request after timeout	Idempotency policy prevents duplicate update where request identifiers are supported

4.5 Adequacy and conclusion

The test set should cover each valid/invalid partition, all quantity and threshold boundaries, both stock-update directions, alert generation and suppression rules, and transaction/concurrency failures. Logs are useful clues, but a successful transaction alone does not prove that stock and alert side effects are correct.

Question 5 – Reverse Engineering a Library Management Executable from Resource Profiles

5.1 Interpretation limits

The assessment refers to a compiled .exe and CPU/memory stress profile but does not attach the executable or profile. Resource spikes alone cannot prove which source-level loop or decision caused them. They can identify time windows for targeted investigation; profiling, symbols, logs, and repeatable workload experiments are needed to confirm causes.

5.2 Conceptual functional/control-flow model

Illustrative workload: search a catalogue, retrieve matching records, check availability, and optionally issue a book. This model is a hypothesis only.

START → Validate request → [Valid?]
No → Return validation error → END
Yes → Load catalogue → For each record:
[Record matches?] — No → next record
Yes → [Available?]
No → record unavailable result
Yes → add to result list
[More records?] — Yes → loop
No → Sort/format results → Return response → END

5.3 Resource spikes and candidate causes

Observed profile pattern (if found)	Possible explanation	How to validate
CPU spike grows with catalogue size	Repeated full scan, expensive matching, or nested loop	Run same query with 1k/10k/100k records; compare CPU time
Memory rises with result count	Unbounded result accumulation or duplicated objects	Measure peak memory for small/large result sets
Long pause on one query type	Sorting, regex matching, file I/O, or lock contention	Profile by function; compare query categories
CPU remains high after response	Background loop, retry storm, or thread not terminating	Capture thread/process traces and idle baseline
Memory grows after repeated searches	Leak or retained cache/result references	Repeat identical workload and measure memory after cleanup

5.4 Reconstructed path and stress tests

ID	Workload/path	Expected observation
LIB-01	Empty search term / invalid request	Fast validation failure; bounded CPU and memory
LIB-02	Search with no matches	Loop completes and returns empty result
LIB-03	Search with one match	Matching branch and result construction execute
LIB-04	Many matches	All valid results returned within agreed response-time/memory limits
LIB-05	Catalogue at maximum supported size	No crash; resource usage remains within documented limit
LIB-06	Repeated searches for same term	No unexplained monotonic memory growth
LIB-07	Slow storage or database response	Timeout/retry policy terminates predictably
LIB-08	Concurrent search and issue operations	No deadlock, inconsistent availability, or resource starvation
LIB-09	Malformed or unusually long search term	Safe handling; no crash or excessive CPU from pathological input
LIB-10	Cancellation/shutdown during large scan	Loop and worker terminate cleanly

5.5 Stability impact and adequacy

- Measure baseline, average, peak, and percentile response time; CPU utilization; peak memory; crash rate; and throughput.
- Use a controlled workload matrix varying catalogue size, match rate, query complexity, and concurrency.
- Map each spike to timestamps and workload IDs; use a profiler or instrumentation to identify functions where possible.
- Test recovery after stress: memory should stabilize, handles/threads should be released, and subsequent requests should still succeed.
- Define acceptance thresholds before testing; do not invent a universal CPU or memory limit when the product specification is absent.

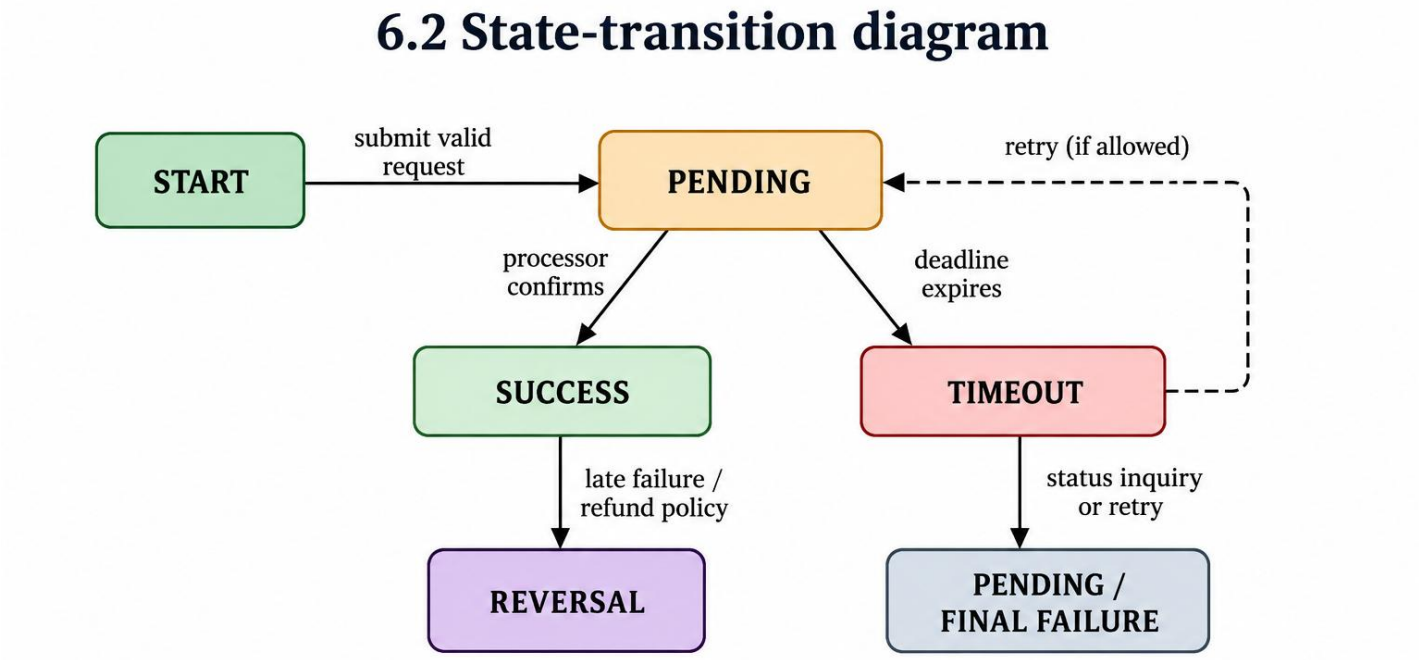
A conceptual CFG helps organize path tests, but a compiled binary and resource chart do not expose a unique CFG. Source code, symbols, disassembly, or instrumented traces are needed for a defensible structural model.

Question 6 – Mobile Payment Gateway State-Transition Testing

6.1 State model and assumptions

The question names Pending, Success, Reversal, and Timeout states but supplies no actual JSON responses or transition rules. The following state machine is an illustrative model. Confirm the legal transitions against the gateway specification before using it as an oracle.

6.2 State-transition diagram



Illustrative Constraints

1. **Terminal states** must not silently return to **Pending**.
2. A **successful payment may be reversed** only through an explicit reversal/refund rule.
3. **Duplicate callbacks** must be **idempotent** (i.e., repeated notifications should not create duplicate transactions or inconsistent state changes).

6.3 State and transition definitions

Current state	Event/trigger	Illustrative next state	Validation
No transaction	Valid request accepted	Pending	Unique transaction ID and required fields exist
Pending	Processor confirms payment	Success	Amount/currency/transaction ID match original request
Pending	Processor declines or processing fails	Reversal or failure outcome	Exact mapping depends on API contract
Pending	Deadline exceeded without final response	Timeout	Timeout response must not imply payment definitely failed if outcome is unknown
Timeout	Status inquiry finds	Success	Same transaction ID; no

	completed payment		duplicate debit
Timeout	Status inquiry confirms failure and reversal completes	Reversal/failure terminal state	Consistent ledger and final status
Success	Duplicate success callback	Success (idempotent)	No second debit or second receipt
Success	Authorized reversal/refund event	Reversal	Ledger records compensating entry
Reversal	Late success callback for original request	Remain Reversal or reconciliation state	Must not resurrect transaction without defined policy
Success	Invalid transition request	Reject	State remains unchanged; audit event recorded

6.4 Comprehensive state-based test suite

ID	Test	Expected result
PAY-01	Submit valid request	Pending response with transaction ID
PAY-02	Pending receives success callback	State becomes Success exactly once
PAY-03	Pending reaches deadline	Timeout state/response follows contract
PAY-04	Timeout then status inquiry reports paid	Reconcile to Success without duplicate debit
PAY-05	Success callback delivered twice	Idempotent result; one ledger debit
PAY-06	Authorized reversal after Success	Reversal recorded with linked original transaction
PAY-07	Attempt Reversal → Pending	Rejected unless explicitly allowed by specification
PAY-08	Attempt Success → Pending	Rejected; terminal-state rule enforced
PAY-09	Callback contains mismatched amount/currency	Rejected or quarantined; no state mutation
PAY-	Unknown transaction ID	Safe error; no transaction created

10		unintentionally
PAY-11	Malformed JSON or missing required field	Validation error; no payment processing
PAY-12	Late success callback after timeout	Reconciliation policy applied; final ledger remains consistent

6.5 Implicit technique and completeness

The technique is state-transition testing: test states, legal transitions, illegal transitions, transition sequences, and events that trigger transitions. A transition coverage matrix should show every defined transition has at least one test. Add transition-pair coverage for important two-step sequences (for example Pending→Timeout followed by Timeout→Success). Include data constraints such as positive amount, supported currency, unique transaction ID, and matching callback values.

Timeout is often an observation that the caller did not receive a definitive result, not proof that the payment processor failed. The system should reconcile status safely rather than automatically repeat a potentially successful payment.

Question 7 – Fuzzing and Boundary Testing of an Encrypted Communication Module

7.1 Decomposition and safety scope

The question describes black-box encrypted outputs and asks for fuzzing and BVA. No actual parameter schema, buffer sizes, cipher mode, or outputs are provided. The proposed tests below are generic and should be run only against an authorized test build or sandbox.

- Input parser: field presence, type, encoding and length.
- Validation layer: allowed character sets, ranges, required fields and cross-field constraints.
- Serialization/buffer handling: length calculation, truncation, padding and encoding conversion.
- Cryptographic layer: key/nonce/IV handling, algorithm configuration, authentication/tag verification.
- Output/error layer: ciphertext format, safe errors, logging and resource cleanup.

7.2 Fuzzing strategy and boundaries

Input dimension	Test values/patterns	Expected safe behavior
Required field	Missing, null, empty, whitespace-only	Reject clearly; no crash
Length	0, 1, max−1, max, max+1, very large	Accept only specified range; no overwrite or unbounded allocation
Encoding	Valid UTF-8, invalid byte sequences, multibyte text	Defined validation; no truncation ambiguity
Type	String instead of integer; array/object instead of string	Type error; no unsafe coercion
Special characters	Quotes, delimiters, control characters,	Treated as data or rejected per

	null byte	schema; no parser confusion
Cross-field constraints	Key/nonce length mismatch; missing authentication tag	Reject before cryptographic processing
Repeated/random inputs	Seeded random, mutated valid inputs, repeated malformed requests	No crash, leak, hang, or state corruption
Ciphertext input	Truncated, modified, oversized, invalid tag	Authentication failure; plaintext not released

7.3 Reconstructed test cases

ID	Input / action	Expected result
ENC-01	All fields valid at minimum supported lengths	Successful encryption and valid output format
ENC-02	Each required field omitted one at a time	Validation failure; no partial output
ENC-03	Field length at max−1, max, max+1	Boundary behavior matches specification
ENC-04	Very large payload within authorized test environment	Graceful size rejection or bounded processing
ENC-05	Invalid encoding/control bytes	Safe reject or defined canonicalization
ENC-06	Wrong type for each field	Typed validation error; no crash
ENC-07	Malformed key/nonce/IV or tag length	Reject; do not continue with unsafe defaults
ENC-08	Modify one bit of authenticated ciphertext	Integrity verification fails; plaintext not returned
ENC-09	Repeat same request	Behavior follows documented determinism/idempotency policy; no accidental state leak
ENC-10	Rapid burst of malformed inputs	Rate/resource limits work; service remains available

7.4 Why this helps prevent overflow and injection

BVA tests lengths at and around documented limits, which can expose off-by-one allocation and truncation defects. Fuzzing explores unexpected combinations that manual tests may miss. Safe implementation should validate lengths before allocation, use memory-safe or bounds-checked operations, reject malformed encodings consistently, avoid building SQL/shell commands from untrusted fields, and authenticate ciphertext

before releasing plaintext. Testing can find evidence of vulnerabilities but cannot by itself prove the absence of all buffer overflows or injection flaws.

7.5 Adequacy

Track unique crashes, hangs, sanitizer findings, rejected-input correctness, and code/branch coverage if instrumentation is available. Keep reproducible seeds and minimized failing inputs. Confirm both successful valid inputs and rejected invalid inputs; do not infer security from encrypted-looking output alone.

Question 8 – Insurance Premium Calculator: Statement vs Branch/Path Coverage

8.1 Why 100% statement coverage is insufficient

Statement coverage confirms that each executable statement ran at least once. It does not guarantee that each outcome of a compound decision ran. For example, tests can execute the premium calculation statement while never exercising a surcharge or eligibility rejection branch. The assessment provides no actual calculator code, so the example below is illustrative.

8.2 Illustrative logic and CFG

```
if age < 18 OR age > 65:
    return NOT_ELIGIBLE
premium = base_rate
if smoker == TRUE AND risk_score >= 7:
    premium = premium * 1.30
else:
    premium = premium * 1.00
return premium
```

CFG:

START → D1 [age < 18 OR age > 65?]

D1-YES → NOT_ELIGIBLE → END

D1-NO → base premium → D2 [smoker AND risk_score ≥ 7?]

D2-YES → premium × 1.30 → RETURN → END

D2-NO → premium × 1.00 → RETURN → END

8.3 Independent paths and tests

Path	Age	Smoker	Risk score	Expected outcome
I1: eligibility rejected	17	Any	Any	NOT_ELIGIBLE; no premium quote
I2: eligible, surcharge true	40	True	7	Base premium × 1.30
I3: eligible, surcharge false by smoker condition	40	False	9	Base premium unchanged
I4: eligible, surcharge false by risk condition	40	True	6	Base premium unchanged
I5: upper eligibility boundary	65	False	6	Eligible if rule is age > 65; base premium

I6: above upper boundary	66	False	6	NOT_ELIGIBLE

8.4 Branch and condition coverage

For the two top-level decisions, branch coverage requires true and false outcomes for the eligibility decision and surcharge decision. Because the surcharge decision contains two atomic conditions, modified condition/decision coverage (MC/DC), where required by the assurance standard, would show each condition independently affects the decision outcome. Example: keep age eligible and risk score ≥7 while toggling smoker; then keep smoker true and toggle risk score around 7.

8.5 Expanded boundary and calculation tests

ID	Input	Expected result
INS-01	Age 17, 18 and 19	Reject 17; accept 18 and 19 if minimum is inclusive
INS-02	Age 64, 65 and 66	Accept through 65; reject 66 under illustrative rule
INS-03	Eligible age, smoker=true, risk=6	No surcharge
INS-04	Eligible age, smoker=true, risk=7	30% surcharge
INS-05	Eligible age, smoker=false, risk=9	No surcharge
INS-06	Risk score below minimum, above maximum, missing, non-numeric	Validate or reject per specification
INS-07	Base rate zero/negative/very large	Reject invalid configuration or apply documented numeric limits
INS-08	Rounding and currency precision at fractional premium	Use defined rounding rule consistently

8.6 Conclusion

The reconstructed strategy goes beyond statement coverage by forcing every branch outcome and important compound-condition effect. For a loop-free example, the independent paths can be enumerated; for real code, construct the actual CFG and calculate V(G). Full path coverage may be impractical where loops or many independent conditions produce a large or unbounded path set, so the chosen criterion must be stated explicitly.

Question 9 – Cloud File Synchronization and Lost Files

9.1 Reverse-engineering the synchronization logic

The assessment mentions activity logs and lost-file defect reports related to file size, type and network speed, but does not include actual logs. The hypotheses below illustrate how patterns should be investigated rather than assert a confirmed root cause.

Hypothetical pattern	Possible cause / missed class	Tests to distinguish hypotheses
Large files disappear on slow networks	Timeout, partial upload, retry or resume defect	Same file over fast/slow network; interrupt at different percentages
Only one file type is lost	Extension/MIME filtering, path handling or metadata bug	Compare same content with different extensions and names
Zero-byte files do not sync	Empty-file special case or size validation defect	Test 0 B, 1 B and nearby size boundaries
Files with Unicode names disappear	Normalization/encoding/path comparison defect	Unicode, combining marks, emoji, long names
Two devices overwrite a file	Conflict-resolution or versioning defect	Concurrent edits from two clients
Sync reports success but remote file is absent	Premature success acknowledgement or commit-order defect	Verify remote checksum and durable metadata after success

9.2 ECP and BVA dimensions

Dimension	Equivalence classes / boundaries
File size	0 bytes; small valid; typical; near configured maximum; over maximum
File type	Supported; unsupported; extension/MIME mismatch; unknown
Network	Stable fast; slow but connected; intermittent; disconnected; reconnecting
File name/path	Normal; empty; maximum length; Unicode; reserved characters; traversal-like patterns
Sync state	New upload; unchanged file; modified file; delete; rename; conflicting versions
Storage	Sufficient space; near full; full; permission denied

9.3 Reconstructed test suite integrating BVA and stress

ID	Scenario	Expected result
SYNC-01	Sync a zero-byte file	File and metadata are preserved if zero-byte files are supported
SYNC-02	Upload at size limit−1, exactly limit, and limit+1	Boundary follows specification; oversized file is safely rejected or handled
SYNC-03	Upload supported file type	Correct content, metadata and checksum at destination
SYNC-04	Upload unsupported type	Clear rejection or documented policy; source is not deleted
SYNC-05	Slow network during upload	Progress/resume or safe failure; no false success
SYNC-06	Disconnect at 25%, 50%, 99%, then reconnect	Resume/retry produces one correct final file
SYNC-07	Large batch of files and concurrent clients	No silent loss; bounded resources; all items reconciled
SYNC-08	Two devices edit same file offline	Conflict policy preserves versions or clearly resolves conflict
SYNC-09	Rename file to Unicode/long name	Path and metadata remain consistent across devices
SYNC-10	Destination disk becomes full	Upload fails safely; source remains available; retry possible
SYNC-11	Delete/rename while sync is pending	Ordering and tombstone/version logic prevent resurrection or accidental deletion
SYNC-12	Compare source and destination checksums after success	Content checksum matches; mismatch is reported and repaired

9.4 Stress testing and data-loss prevention

- Run mixed workloads across file sizes, types, concurrency levels and network profiles, including repeated interruptions.
- Verify end-to-end integrity using checksums, file counts, version IDs and metadata comparisons.
- Confirm acknowledgements occur only after the system's defined durability point.
- Test retry idempotency and conflict resolution so retries do not duplicate or overwrite valid data.
- Check logs for correlation IDs, retry counts, state transitions and explicit failure reasons.
- Use fault injection for network loss, process restart and storage exhaustion; verify recovery after restart.

Completeness should be reported as coverage of the defined partition matrix and risk scenarios, not as a claim that every possible file and network condition has been tested.

Question 10 – Reverse Engineering E-commerce Search and Filter SQL

10.1 Problem interpretation and assumptions

The assessment describes SQL queries generated by a Search and Filter module, but no queries or result datasets are supplied. The decision table below is an illustrative model. Assume the module can filter by category, minimum price, maximum price, availability and keyword. Actual field names and semantics must be confirmed from the product specification or observed requests.

10.2 Decomposition

- Parameter parsing: missing, duplicated, malformed or unsupported query parameters.
- Filter normalization: convert ranges, categories and keywords into a canonical representation.
- Predicate construction: AND/OR/NOT semantics, grouping and precedence.
- Query execution: parameterized SQL, joins, indexes, ordering and pagination.
- Result formatting: total count, page boundaries, sorting and empty-result behavior.
- Security and performance: SQL injection resistance, query timeouts and bounded resource usage.

10.3 Decision table

Rule	Keyword matches?	Category matches?	Price in range?	In stock?	Expected inclusion
R1	Yes	Yes	Yes	Yes	Include
R2	No	Yes	Yes	Yes	Exclude if keyword is required
R3	Yes	No	Yes	Yes	Exclude if category is required
R4	Yes	Yes	No	Yes	Exclude
R5	Yes	Yes	Yes	No	Exclude when in-stock filter is enabled
R6	Yes	Yes	Yes	No filter requested	Include if stock is not a required condition
R7	Keyword OR category matches	At least one matches	Yes	Any	Include if UI semantics explicitly use OR
R8	Excluded by NOT condition	Otherwise matches	Yes	Yes	Exclude

The table shows why AND/OR/NOT semantics must be explicit. For example, keyword AND category returns only items satisfying both predicates; keyword OR category returns items satisfying either. Parentheses must be preserved when conditions are mixed.

10.4 Reconstructed SQL logic (illustrative)

```
SELECT p.id, p.name, p.price, p.category_id, p.stock
FROM products AS p
WHERE (:keyword_is_empty = TRUE OR p.name LIKE :keyword_pattern)
  AND (:category_is_null = TRUE OR p.category_id = :category_id)
  AND (:min_price_is_null = TRUE OR p.price >= :min_price)
  AND (:max_price_is_null = TRUE OR p.price <= :max_price)
  AND (:in_stock_filter = FALSE OR p.stock > 0)
ORDER BY p.name ASC
LIMIT :page_size OFFSET :offset;
```

This is conceptual parameterized SQL, not a claim about the actual implementation. Real SQL dialects may not support Boolean placeholders in this exact form. Build the query using parameter binding and allow-list sort columns/directions; do not concatenate raw user input into SQL.

10.5 Reconstructed test cases

ID	Search/filter input	Expected result
SRCH-01	No filters	All products allowed by default visibility rules, paginated deterministically
SRCH-02	Keyword that matches one product	Only matching products returned
SRCH-03	Existing category	Products in that category returned
SRCH-04	Keyword AND category	Only products satisfying both conditions returned
SRCH-05	Keyword OR category (if supported)	Products satisfying either condition returned
SRCH-06	NOT a selected brand/category (if supported)	Excluded items absent; remaining filters still applied
SRCH-07	Price exactly at min and max	Inclusive/exclusive behavior matches specification
SRCH-08	min_price > max_price	Validation error or defined empty result; no unexpected broad result
SRCH-09	Malformed price, negative price, huge numeric value	Safe validation; no query crash
SRCH-	Page size 0, negative, maximum,	Bounded and documented pagination

10	maximum+1	behavior
SRCH-11	SQL-injection-like text in keyword	Treated as data; query structure unchanged
SRCH-12	No products match	Empty result and count are consistent
SRCH-13	Sort by allowed field and invalid field	Allowed sorting works; invalid field safely rejected or defaults
SRCH-14	Many filters and large dataset	Correct result semantics and acceptable performance

10.6 Completeness and conclusion

Evaluate completeness by enumerating each filter's on/off state and important value partitions, then cover all meaningful combinations using a decision table. Full combinatorial coverage can grow exponentially; use pairwise or risk-based combinations for low-risk dimensions while preserving exhaustive tests for critical logic such as price boundaries, authorization/visibility, and AND/OR precedence. Validate each expected result against a small, known dataset whose rows make the Boolean rules unambiguous.